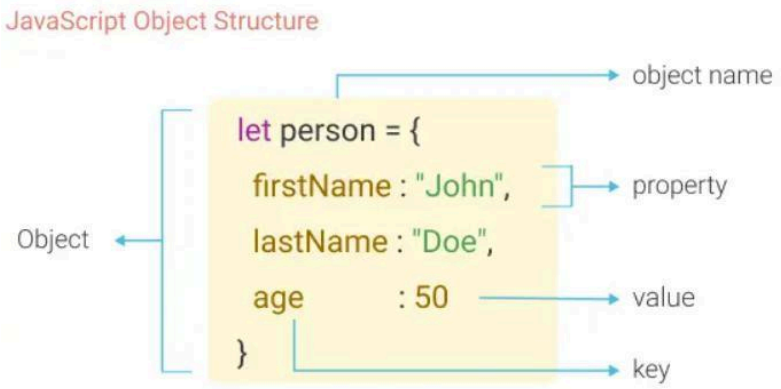
AWT- Important Question Answers from U3 & U4 for MSE-2

1. Show how the objects are created and modified in Javascript with example.

Creation and Modification of Objects in JavaScript

In JavaScript, **objects** are collections of **properties (key–value pairs)**. Properties can store data or functions (methods), and objects are **dynamic**—properties can be added or removed at runtime.

📊 Object Concept




```
let person = {
    firstName: 'John',
    lastName: 'Doe'
};

person.firstName = 'Jane';

console.log(person);

//{ firstName: 'Jane', lastName: 'Doe' }
```

- **1. Creating Objects**

✅ (a) Using new Operator

```
var my_car = new Object();
my_car.make = "Ford";
my_car.model = "Fusion";
```

- Creates an empty object
- Properties are added later

✅ (b) Using Object Literal (Shortcut Method)

```
var my_car = {
  make: "Ford",
  model: "Fusion"
};
```

- Easy and commonly used method

🔹 2. Accessing Object Properties

✔️ Dot Notation

```
var m = my_car.make;
```

✔️ Bracket Notation

```
var m = my_car["make"];
```

🔹 3. Modifying Objects

➤ Adding New Properties

```
my_car.year = 2020;
```

➤ Modifying Existing Properties

```
my_car.model = "Mustang";
```

➤ Adding Nested Object

```
my_car.engine = new Object();
my_car.engine.hp = 263;
```

🔹 4. Deleting Properties

```
delete my_car.model;
```

🔹 5. Iterating Properties

```
for (var prop in my_car) {
```

```
  document.write(prop + ": " + my_car[prop] + "<br>");
}
```

🔹 Key Points

- Objects are **dynamic** → properties can be added/removed anytime
- Properties are accessed using **dot or bracket notation**
- Objects can contain **other objects (nested objects)**

✍️ Conclusion (Exam Ready)

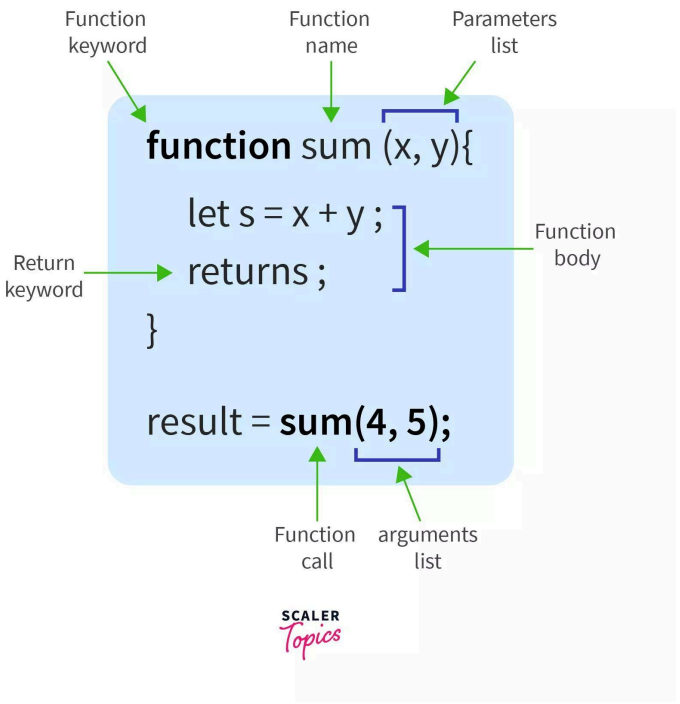
Objects in JavaScript can be created using the new operator or object literals. They are dynamic in nature, allowing properties to be added, modified, or deleted during execution, making them highly flexible.

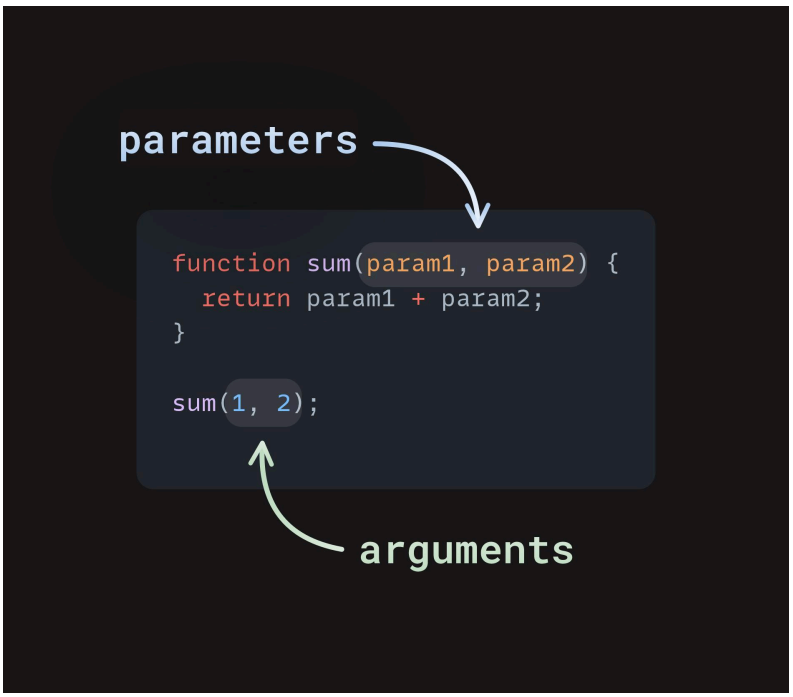
2. Explain how are functions defined and used in Javascript.

Functions in JavaScript – Definition and Usage

A **function** in JavaScript is a block of code designed to perform a specific task. It is executed when it is **called (invoked)**. A function consists of a **header** and a **body**.

📊 Function Concept





🔹 1. Defining a Function

A function is defined using the function keyword.

✅ Syntax:

```
function functionName(parameters) {
  // statements (function body)
}
```

✔️ Example:

```
function greet() {
  document.write("Hello World");
}
```

- function → keyword
- greet → function name
- {} → body of function

🔹 2. Calling (Using) a Function

A function is executed by calling its name followed by parentheses.

```
greet();  // function call
```

🔹 3. Functions with Parameters

- Parameters receive values from the function call
- JavaScript uses **pass-by-value** (values are copied)

✔️ Example:

```
function add(a, b) {
  document.write(a + b);
}

add(5, 3);  // Output: 8
```

🔹 4. Returning Values

Functions can return values using return.

```
function square(x) {
  return x * x;
}

var result = square(4);  // result = 16
```

🔹 5. Functions as Objects

- Functions are **objects in JavaScript**
- They can be assigned to variables

```
function fun() {
  document.write("This is fun!");
}

var ref_fun = fun;
ref_fun();  // calls function
```

🔹 6. Important Points

- Functions must be defined **before they are called**
- Parameters are optional
- Functions can be reused multiple times
- JavaScript functions are flexible and dynamic

✍️ Conclusion (Exam Ready)

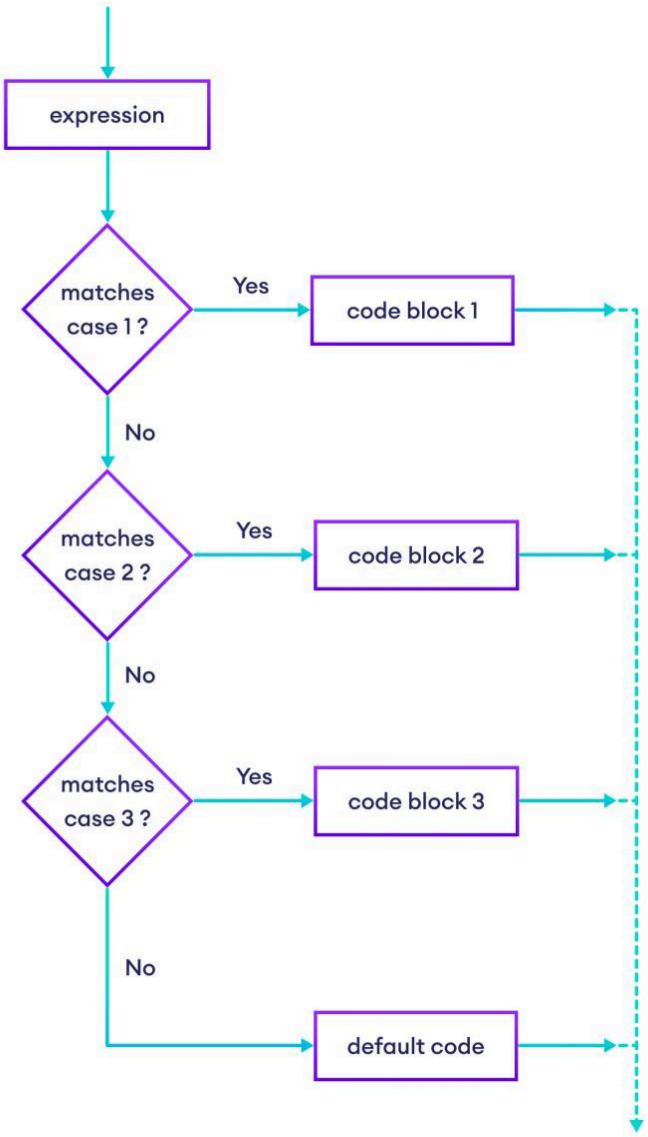
Functions in JavaScript are defined using the function keyword and are used to perform reusable tasks. They can accept parameters, return values, and are treated as objects, making them highly flexible.

3. Explain control statements in Javascript with example.

Control Statements in JavaScript

Control statements are used to control the **flow of execution** of a program. They decide which statements are executed, how many times, and under what conditions.

📊 Control Flow Concept



🔹 Types of Control Statements

1️⃣ Selection Statements (Decision Making)

Used to choose between alternatives.

✔️ (a) if Statement

```
if (a > b) {
   document.write("a is greater");
}
```

✔️ (b) if-else Statement

```
if (a > b) {
   document.write("a is greater");
} else {
   document.write("b is greater");
}
```

✔️ (c) switch Statement

```
var day = 2;
switch(day) {
   case 1: document.write("Monday"); break;
   case 2: document.write("Tuesday"); break;
   default: document.write("Invalid");
}
```

2️⃣ Looping Statements (Iteration)

Used to repeat a block of code.

✔️ (a) while Loop

```
var i = 1;
while (i <= 3) {
   document.write(i);
   i++;
}
```

✔️ (b) for Loop

```
for (var i = 1; i <= 3; i++) {
   document.write(i);
}
```

✔️ (c) do-while Loop

```
var i = 1;
do {
   document.write(i);
```

```
  i++;
} while (i <= 3);
```

3️⃣ Control Expressions

- Conditions are evaluated as **true or false**
- JavaScript treats:
  - 0, null, undefined → false
  - Non-zero values → true

🔹 Key Points

- Control statements guide program execution
- Selection → decision making
- Loops → repetition
- Conditions are based on Boolean values

✍️ Conclusion (Exam Ready)

Control statements in JavaScript are used to control the flow of execution. They include selection statements like if and switch, and looping statements like for and while, which help in decision-making and repetition.

4. Write a Javascript function to find factorial of a given number.

JavaScript Function to Find Factorial

The **factorial** of a number n is calculated as:
[
n! = n \times (n-1) \times (n-2) \times ... \times 1
]

🔹 Using Function (Loop Method)

✅ Program:

```
function factorial(n) {
  var fact = 1;

  for (var i = 1; i <= n; i++) {
    fact = fact * i;
  }

  return fact;
```

```
}

// Example usage
var result = factorial(5);
document.write("Factorial is: " + result);
```

🔹 Output

Factorial is: 120

🔹 Using Recursion (Alternative)

```
function factorial(n) {
  if (n == 0 || n == 1)
    return 1;
  else
    return n * factorial(n - 1);
}
```

🔹 Key Points

- Uses **function definition and call**
- Loop or recursion can be used
- Factorial of 0 is **1**

✍️ Conclusion (Exam Ready)

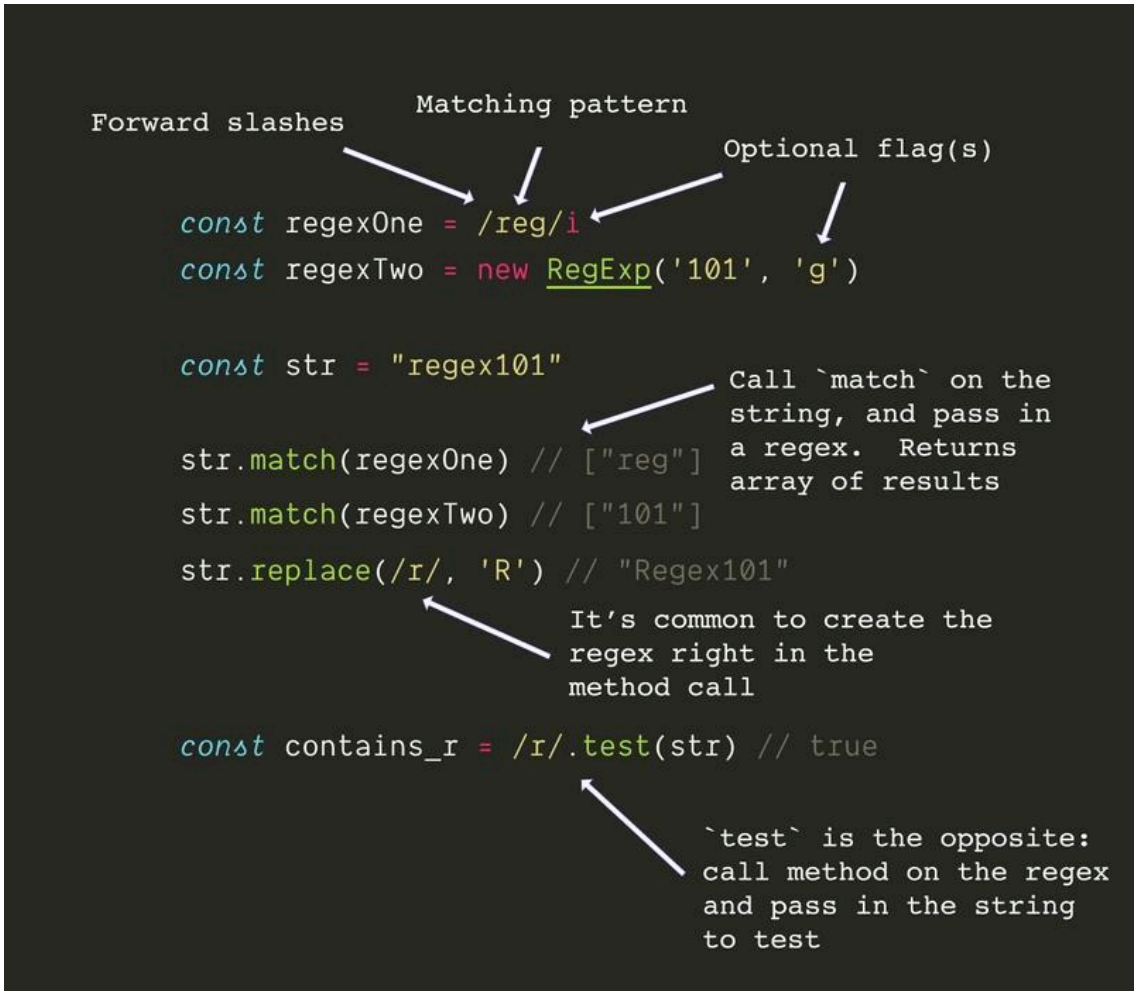
A JavaScript function can be used to compute factorial using either loops or recursion. It multiplies all numbers from 1 to the given number to produce the result.

5. Demonstrate pattern matching in javascript

Pattern Matching in JavaScript

Pattern matching in JavaScript is done using **Regular Expressions (RegExp)**. It is used to **search, match, validate, and replace patterns** in strings.

📊 Pattern Matching Concept



🔹 Regular Expression Basics

- Patterns are written between **slashes** / /
- Example:

/abc/

🔹 1. Using search() Method

- Finds position of pattern in string
- Returns index or -1 if not found

var str = "Rabbits are furry";
var pos = str.search(/bits/);

document.write(pos); // Output: 3

🔹 2. Using match() Method

- Returns matched values as an array

```
var str = "Having 4 apples and 3 oranges";
var result = str.match(/\d/g);

document.write(result);  // Output: 4,3
```

🔹 3. Using replace() Method

- Replaces matched pattern with new value

```
var str = "Fred, Freddie";
var newStr = str.replace(/Fre/g, "Boy");

document.write(newStr);  // Boyd, Boyddie
```

🔹 4. Using split() Method

- Splits string based on pattern

```
var str = "grapes:apples:oranges";
var arr = str.split(":");

document.write(arr);  // grapes,apples,oranges
```

🔹 Common Pattern Symbols

- \d → digit
- \w → word character
- . → any character
- * → zero or more
- + → one or more
- ^ → start of string
- $ → end of string

🔹 Example: Phone Number Validation

```
function check(num) {
  var ok = num.search(/^\d{3}-\d{4}$/);

  if (ok == 0)
    document.write("Valid number");
```

```
  else
    document.write("Invalid number");
}

check("444-5432");
```

✍️ Conclusion (Exam Ready)

Pattern matching in JavaScript is performed using regular expressions. Methods like search(), match(), replace(), and split() are used to find and manipulate patterns in strings efficiently.

6. How can you change the visibility of an element using Javascript.

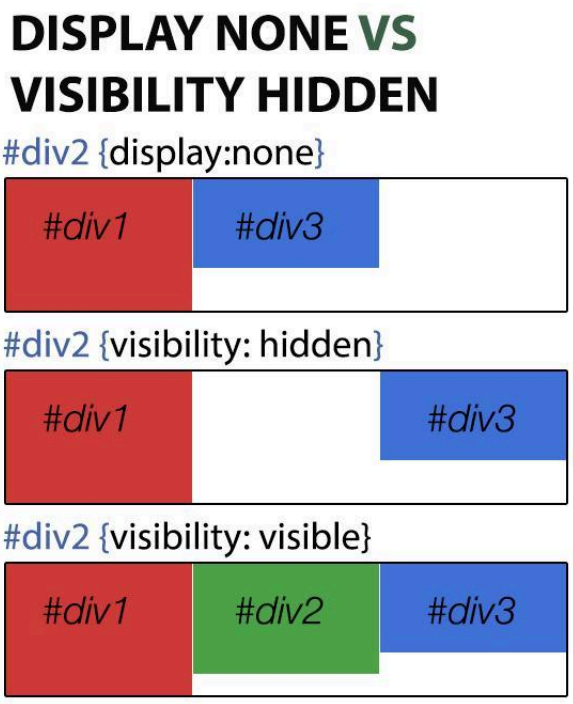
Changing Visibility of an Element using JavaScript

In JavaScript, the **visibility of an HTML element** can be changed using the **CSS visibility property** or by modifying it through the DOM.

According to your notes, the visibility property has two values:

- **visible** → element is shown
- **hidden** → element is not displayed

📊 Concept



🔹 Method to Change Visibility

Use JavaScript to modify the style property:

```
document.getElementById("myDiv").style.visibility = "hidden";
```

🔹 Example Program

```
<!DOCTYPE html>
<html>
<body>

<p id="text">Hello World</p>

<button onclick="hideText()">Hide</button>
<button onclick="showText()">Show</button>

<script>
function hideText() {
  document.getElementById("text").style.visibility = "hidden";
}

function showText() {
  document.getElementById("text").style.visibility = "visible";
}
</script>

</body>
</html>
```

🔹 Toggle Visibility Example

```
function toggle() {
  var x = document.getElementById("text");

  if (x.style.visibility === "hidden")
    x.style.visibility = "visible";
  else
    x.style.visibility = "hidden";
}
```

🔹 Key Points

- Uses **DOM access (getElementById)**
- Uses **style.visibility property**
- Controlled dynamically using JavaScript events

✍️ Conclusion (Exam Ready)

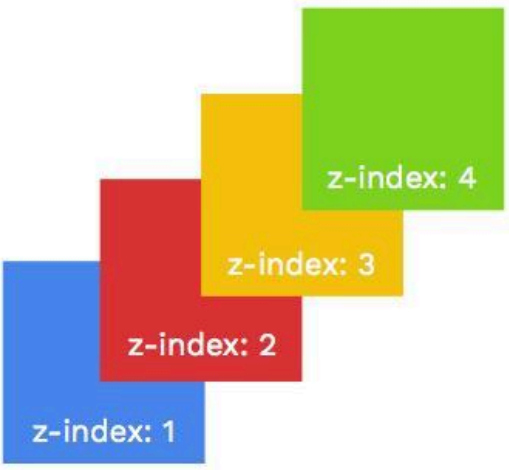
The visibility of an element can be changed in JavaScript by modifying the visibility property through the DOM. It allows elements to be shown or hidden dynamically based on user interaction.

7. Describe how can you achieve stacking of elements.

Stacking of Elements in JavaScript / CSS

Stacking of elements refers to placing HTML elements **on top of each other** (overlapping) to create a layered (front–back) effect on a web page.

📊 Stacking Concept



🔹 How Stacking is Achieved

1️⃣Using Positioning

- Elements must have position:
  - absolute

- relative

Box 1
Box 2

👉 These elements will **overlap**

2️⃣ Using z-index Property

- Controls which element appears **on top**
- Higher z-index → appears above lower values

Box 1
Box 2

👉 Box 2 appears above Box 1

3️⃣ Using JavaScript

JavaScript can dynamically change stacking:

document.getElementById("box1").style.zIndex = "5";

🔹 Example

<!DOCTYPE html>
<html>
<body>

</body>
</html>

👉 Blue box appears on top

🔹 Key Points

- Overlapping requires **positioning (absolute/relative)**

- z-index controls **stack order**
- Higher value = higher layer
- JavaScript can dynamically change stacking

✍️ Conclusion (Exam Ready)

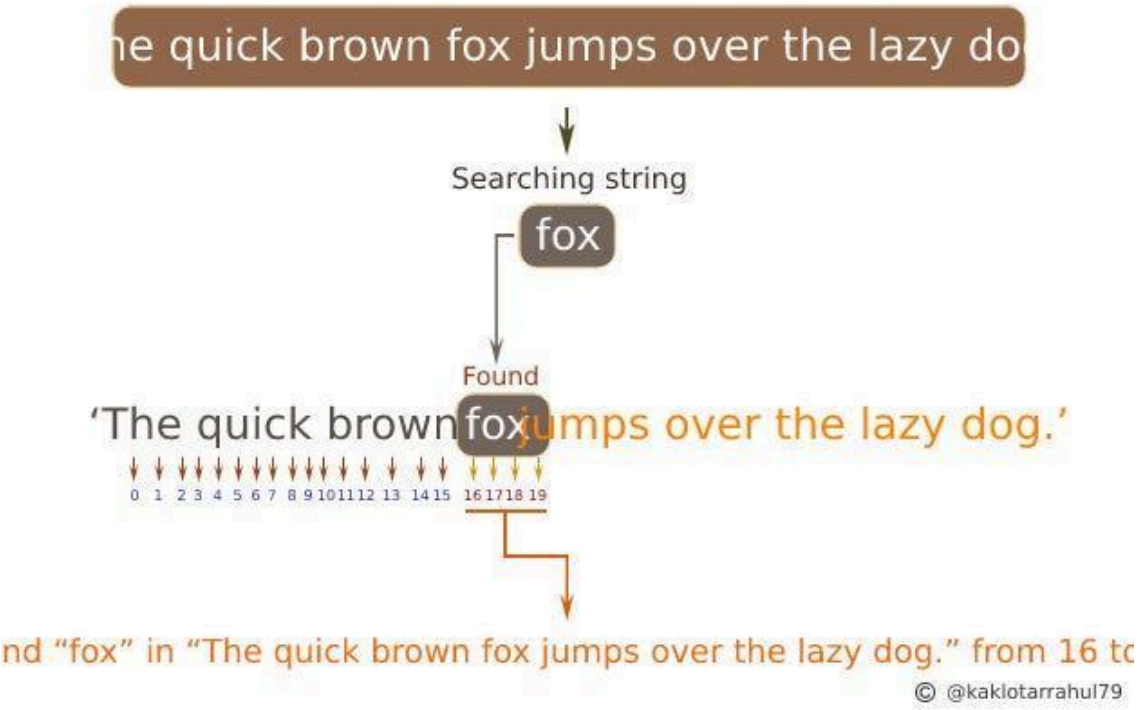
Stacking of elements is achieved using CSS positioning and the z-index property. It allows elements to overlap, with higher z-index values appearing above lower ones, creating a layered visual effect.

8. Compare search() and match() methods provided by String object in Javascript for pattern matching.

Comparison of search() and match() Methods in JavaScript

Both **search()** and **match()** methods are used for **pattern matching** with regular expressions in JavaScript, but they differ in **output and usage**.

📊 Concept



🔍 Differences Between search() and match()

Feature	search()	match()
Purpose	Finds position of pattern	Finds actual matches

Return Value	Index of first match or -1	Array of matched values
Multiple Matches	Not supported	Supported (with g flag)
Output Type	Number	Array
Use Case	Checking existence/position	Extracting values

🔹 1. search() Method

- Returns **index of first occurrence**
- Returns -1 if not found

✅ **Example:**

```
var str = "Rabbits are furry";
var pos = str.search(/bits/);

document.write(pos);  // Output: 3
```

🔹 2. match() Method

- Returns **matched values as array**
- Can return multiple matches

✅ **Example:**

```
var str = "Having 4 apples and 3 oranges";
var result = str.match(/\d/g);

document.write(result);  // Output: 4,3
```

🔹 Key Points

- search() → gives **position**
- match() → gives **actual matched data**
- match() is more powerful for extracting multiple values

✍️ Conclusion (Exam Ready)

The search() method returns the index of the first match, while the match() method returns the matched values. match() is more useful when extracting multiple pattern matches, whereas search() is used to find the position of a pattern.

9. Illustrate a program in Javascript to validate the email ID.

JavaScript Program to Validate Email ID

Email validation in JavaScript is commonly done using **regular expressions (pattern matching)** to check whether the entered email follows a valid format (like name@domain.com).

🔹 Example Program

```
<!DOCTYPE html>
<html>
<body>

Enter Email: <input type="text" id="email">
<button onclick="validate()">Validate</button>

<script>
function validate() {
  var email = document.getElementById("email").value;

  // Regular expression pattern
  var pattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;

  if (pattern.test(email)) {
    alert("Valid Email ID");
  } else {
    alert("Invalid Email ID");
  }
}
</script>

</body>
</html>
```

🔹 Explanation of Pattern

```
^[a-zA-Z0-9._%+-]+   → Username (letters, numbers, symbols)
@                    → Must contain @
[a-zA-Z0-9.-]+       → Domain name
```

\. → Dot (.)
[a-zA-Z]{2,} → Extension (com, in, org, etc.)
$

🔹 Key Points

- Uses **regular expressions (RegExp)**
- Checks format: **username@domain.extension**
- Ensures valid structure before submission

✍️ Conclusion (Exam Ready)

Email validation in JavaScript is performed using regular expressions to ensure the email follows a correct format. It helps prevent invalid data entry in web forms.

10. Illustrate the following with suitable code snippets.

i. getElementById()
ii. getElementByClassName()

i. getElementById()

- Used to select an element using its **id**

```
<!DOCTYPE html>
<html>
<body>

<p id="demo">Hello World</p>
<button onclick="changeText()">Click</button>

<script>
function changeText() {
  document.getElementById("demo").innerHTML = "Text Changed";
}
</script>

</body>
</html>
```

ii. getElementsByClassName()

- Used to select elements using **class name** (returns a collection)

```
<!DOCTYPE html>
<html>
<body>

<p class="test">First</p>
<p class="test">Second</p>
<button onclick="changeColor()">Click</button>

<script>
function changeColor() {
  var x = document.getElementsByClassName("test");
  for (var i = 0; i < x.length; i++) {
    x[i].style.color = "red";
  }
}
</script>

</body>
</html>
```

11. Use Javascript to set the right position of a partitioned element.

JavaScript to Set the Right Position of a Positioned Element

To set the **right position** of an element, JavaScript modifies the CSS **right property** of an element whose position is set to absolute or relative.

✅ Example

```
<!DOCTYPE html>
<html>
<body>

<div id="box" style="position:absolute; right:0px; width:100px; height:100px;
background:red;">
Box
</div>

<button onclick="moveRight()">Move Right</button>
```

```
<script>
function moveRight() {
  document.getElementById("box").style.right = "100px";
}
</script>

</body>
</html>
```

✍️ Explanation

- position:absolute → allows element to be positioned
- style.right → sets distance from the **right side of the page**
- JavaScript dynamically changes the position when button is clicked

✔️ Conclusion

The right position of an element can be set in JavaScript by modifying the style.right property of a positioned element.

12. Explain Implicit and Explicit type conversion in javascript with examples.

Implicit and Explicit Type Conversion in JavaScript

Type conversion means **changing a value from one data type to another**. JavaScript supports two types:

🔹 1. Implicit Type Conversion (Type Coercion)

- Conversion is done **automatically by JavaScript interpreter**
- Happens during operations like +, *, etc.

✅ Examples:

```
var x = "5" + 2;
document.write(x);   // Output: "52" (number converted to string)

var y = "5" * 2;
document.write(y);   // Output: 10 (string converted to number)
```

👉 JavaScript automatically converts types based on operation

🔹 2. Explicit Type Conversion

- Conversion is done **manually by programmer**
- Uses functions like Number(), String(), etc.

✅ **Examples:**

```
var a = "123";
var num = Number(a);   // string → number

var b = 10;
var str = String(b);   // number → string

document.write(num + 1);  // 124
document.write(str + 1);  // "101"
```

🔹 Key Difference

Feature	Implicit Conversion	Explicit Conversion
Done By	JavaScript automatically	Programmer manually
Control	Less control	Full control
Example	"5" + 2	Number("5")

✍️ Conclusion (Exam Ready)

Implicit conversion is performed automatically by JavaScript during operations, while explicit conversion is done manually using conversion functions like Number() and String().

13. Describe different dialog boxes in javascript (alert, confirm, prompt) with syntax and usage.

JavaScript Dialog Boxes

JavaScript provides three built-in dialog boxes for user interaction:
alert(), confirm(), and prompt(). These are methods of the **window object**.

🔹 1. alert() Dialog Box

- Displays a **message with an OK button**
- Used to show information to the user

✅ Syntax:

```
alert("message");
```

✔️ Example:

```
alert("Welcome to JavaScript");
```

🔹 2. confirm() Dialog Box

- Displays a message with **OK and Cancel buttons**
- Returns **true (OK)** or **false (Cancel)**

✅ Syntax:

```
confirm("message");
```

✔️ Example:

```
var result = confirm("Do you want to continue?");
if (result)
  alert("You pressed OK");
else
  alert("You pressed Cancel");
```

🔹 3. prompt() Dialog Box

- Displays a message with **input field + OK/Cancel**
- Returns the **entered value**

✅ Syntax:

```
prompt("message", "default value");
```

✔️ Example:

```
var name = prompt("Enter your name:", "");
alert("Hello " + name);
```

🔹 Key Points

- alert() → display message
- confirm() → get user decision (true/false)
- prompt() → take user input

✍️ Conclusion (Exam Ready)

JavaScript provides three dialog boxes—alert, confirm, and prompt—for user interaction. They are used to display messages, get user confirmation, and accept input respectively.

14.What are arrays in Javascript ? Explain different array methods with example.

Arrays in JavaScript

In JavaScript, **arrays are objects** that store a collection of values (elements). Elements can be of **different data types**, and array indexing starts from **0**.

🔹 Creating an Array

```
var arr = [1, 2, 3, 4];
```

or

```
var arr = new Array(1, 2, 3, 4);
```

🔹 Different Array Methods with Examples

1️⃣join()

- Converts array elements into a string

```
var a = ["A", "B", "C"];
document.write(a.join("-"));   // A-B-C
```

2️⃣reverse()

- Reverses the array

```
var a = [1, 2, 3];
a.reverse();
document.write(a);   // 3,2,1
```

3️⃣sort()

- Sorts elements (as strings by default)

```
var a = [3, 1, 2];
a.sort();
document.write(a);   // 1,2,3
```

4️⃣concat()

- Combines arrays

```
var a = [1, 2];
var b = [3, 4];
var c = a.concat(b);
document.write(c);   // 1,2,3,4
```

5️⃣ slice()

- Returns part of an array

```
var a = [10, 20, 30, 40];
var b = a.slice(1, 3);
document.write(b);   // 20,30
```

6️⃣ push() and pop()

- Add/remove elements at end

```
var a = [1, 2];
a.push(3);   // [1,2,3]
a.pop();     // [1,2]
```

7️⃣ shift() and unshift()

- Remove/add elements at beginning

```
var a = [2, 3];
a.unshift(1);   // [1,2,3]
a.shift();      // [2,3]
```

🔹 Key Points

- Arrays are **dynamic and flexible**
- Can store **mixed data types**
- Provide many built-in methods for manipulation

✍️ Conclusion (Exam Ready)

Arrays in JavaScript are objects used to store multiple values. Methods like join(), reverse(), sort(), concat(), and slice() help in manipulating array data efficiently.

15.Create an HTML document to illustrate the concept of dynamic stacking of image . three images are placed on the document so that they overlap, when a user clicks on any image , it must come to the top position.

HTML + JavaScript Program for Dynamic Stacking of Images

```
<!DOCTYPE html>
<html>
<head>
<title>Dynamic Image Stacking</title>

<style>
img {
  position: absolute;
  width: 150px;
  height: 150px;
  cursor: pointer;
}
#img1 { top: 50px; left: 50px; z-index: 1; }
#img2 { top: 80px; left: 80px; z-index: 2; }
#img3 { top: 110px; left: 110px; z-index: 3; }
</style>

<script>
var topIndex = 3;

function bringToFront(img) {
  topIndex++;
  img.style.zIndex = topIndex;
}
</script>

</head>

<body>

<img src="image1.jpg" id="img1" onclick="bringToFront(this)">
<img src="image2.jpg" id="img2" onclick="bringToFront(this)">
<img src="image3.jpg" id="img3" onclick="bringToFront(this)">

</body>
</html>
```

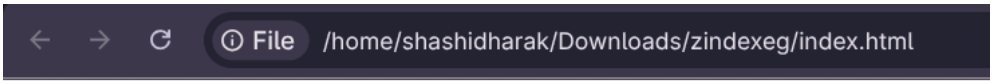
✍️ Explanation

- Images are positioned using **absolute positioning** so they overlap
- z-index controls stacking order
- When an image is clicked, z-index is increased using JavaScript
- The clicked image moves to the **top layer**

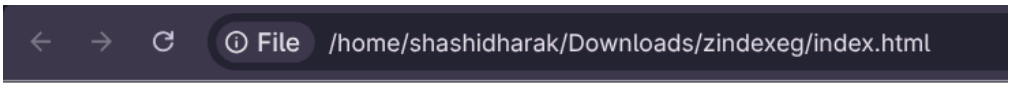
✔️ Conclusion

Dynamic stacking is achieved by modifying the z-index of overlapping images using JavaScript, allowing the clicked image to come to the top.

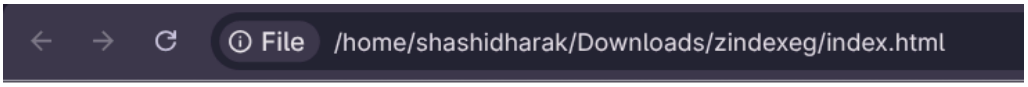
Output:



I click on image 2



I click on image 1



MCA 'B' Section Important Questions

1. Event and event handling

1. What is an Event?

An **event** is a signal that something has happened in the browser or due to user interaction.
It can be:

- User actions → clicking a button, typing in input
- Browser actions → page load, page unload

👉 Example: Clicking a button triggers a **click event**.

---

2. What is Event Handling?

Event handling is the process of writing code that runs automatically when an event occurs.

- The code that runs is called an **event handler**
- It is executed **only when the event happens**

👉 Example:

<button onclick="alert('Clicked!')">Click Me</button>

Here, onclick is the event and alert() is the handler.

---

3. Event-Driven Programming

JavaScript uses **event-driven programming**, where:

- Program execution is **not sequential**
- Code runs **only when events occur**

👉 Events can happen at unpredictable times (user actions, browser behavior).

---

4. Event Handler

An **event handler** is a function/script that is executed when an event occurs.

👉 Example:

```
function greet() {
  alert("Hello!");
}

<button onclick="greet()">Click</button>
```

---

5. Event Registration

The process of linking an event to a handler is called **event registration**.

There are **two ways**:

(a) Using HTML attributes

```
<button onclick="myFunction()">Click</button>
```

(b) Using JavaScript (DOM)

```
document.getElementById("btn").onclick = myFunction;
```

---

6. Common Events

Some frequently used events:

Event	Description
onclick	When element is clicked
onload	When page loads
onunload	When page closes
onmouseover	Mouse enters element
onmouseout	Mouse leaves element
onsubmit	Form submission
onchange	Value changes

---

7. Example: Button Click Event

```
<input type="button" value="Click Me" onclick="showMsg()">

<script>
function showMsg() {
  alert("Button Clicked!");
}
</script>
```

---

8. Example: Page Load Event

```
<body onload="welcome()">

<script>
function welcome() {
  alert("Welcome to the page!");
}
</script>
```

---

9. Important Points

- Events are **case-sensitive** (use lowercase like `click`, not `Click`)
- Event handlers should **not use** `document.write()`
- Events help in **user interaction and validation**

---

10. jQuery Events (Extra from your second file)

jQuery simplifies event handling:

```
$("button").click(function(){
  alert("Clicked!");
});
```

Some jQuery event methods:

- click()
- dblclick()
- mouseenter()
- mouseleave()
- focus()
- blur()

Final Summary (Exam Ready)

- An **event** is an occurrence like a click or page load
- **Event handling** is executing code in response to events
- Uses **event handlers** (functions)
- Works on **event-driven programming**
- Events can be registered using HTML attributes or JavaScript
- Widely used for **user interaction, validation, and dynamic web pages**

2. DOM (Document Object Model)

1. What is DOM?

The **Document Object Model (DOM)** is a programming interface (API) that represents an HTML or XML document as a **tree structure of objects**.

- Each HTML element becomes an **object (node)**
- JavaScript can **access, modify, add, or delete** elements using DOM

👉 It acts as a bridge between **HTML documents and JavaScript programs**

---

2. Structure of DOM

The DOM represents a document like a **tree (hierarchical structure)**:

```
Document
 └── html
    ├── head
    └── body
       ├── h1
       └── p
```

- Root → document
- Parent/Child relationships exist
- Each element is a **node**

👉 Documents in DOM have a **tree-like structure**

---

3. DOM Objects

In JavaScript:

- Every HTML element is treated as an **object**
- Objects have:
    - **Properties** (data)
    - **Methods** (functions)

👉 Example:

<input type="text" id="name">

document.getElementById("name").value = "Shashi";

---

4. Important DOM Objects

- **Window object** → Represents browser window
- **Document object** → Represents HTML page
- **Form elements** → buttons, inputs, etc.

👉 Every window has a **document object** inside it

---

5. Accessing Elements in DOM

There are multiple ways:

(a) Using index (old DOM 0)
document.forms[0].elements[0];

(b) Using name
document.myForm.myButton;

(c) Using ID (most common)
document.getElementById("idName");

👉 getElementById() is part of DOM Level 1

---

6. DOM Manipulation

Using DOM, we can:

- Change content
- Change style
- Add/remove elements

👉 Example:

document.getElementById("demo").innerHTML = "Hello World!";

---

7. DOM Levels

- **DOM 0** → Basic model (early browsers)
- **DOM 1** → Standard structure (HTML & XML)
- **DOM 2** → Added styles and events
- **DOM 3** → Advanced features (validation, formatting)

---

8. Features of DOM

- Platform independent
- Language independent
- Supports dynamic web pages
- Allows real-time updates

---

9. Example Program

```
<!DOCTYPE html>
<html>
<body>

<p id="demo">Original Text</p>

<button onclick="changeText()">Click</button>

<script>
function changeText() {
  document.getElementById("demo").innerHTML = "Text Changed!";
}
</script>

</body>
</html>
```

---

10. Final Exam Answer (Short)

- DOM is an **API that represents HTML documents as objects**
- It allows **JavaScript to manipulate web pages dynamically**
- Structure is **tree-like**
- Elements can be accessed using **ID, name, or index**
- Supports **event handling and dynamic changes**

3. JQuery

1. What is jQuery?

jQuery is a lightweight JavaScript library designed to make web development easier by simplifying tasks like:

- HTML manipulation
- Event handling
- Animations
- AJAX calls

👉 It follows the idea: **"Write less, do more"**

---

2. Why use jQuery?

- Reduces long JavaScript code into simple methods
- Easy DOM manipulation
- Handles events easily
- Cross-browser compatibility

---

3. Adding jQuery to a Web Page

You can include jQuery using a <script> tag:

<script src="jquery-3.7.1.min.js"></script>

Or using CDN:

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

---

4. jQuery Syntax

Basic syntax:

$(selector).action()

- $ → jQuery
- selector → selects HTML element
- action() → operation to perform

👉 Example:

$("p").hide();

---

5. Document Ready Event

Ensures code runs after page loads:

$(document).ready(function(){

```
  // jQuery code
});
```

Shortcut:

```
$(function(){
  // code
});
```

👉 Prevents code from running before page is ready

---

6. jQuery Selectors

Used to select elements:

Selector	Example	Description
Element	$("p")	Select all <p>
ID	$("#id")	Select by id
Class	$(".class")	Select by class

---

7. jQuery Events

Events are user actions like click, hover, etc.

👉 Example:

```
$("p").click(function(){
  $(this).hide();
});
```

Common events:

- click()
- dblclick()

- mouseenter()
- mouseleave()
- focus()
- blur()

---

8. jQuery Effects

jQuery provides built-in effects:

(a) Hide / Show

```
$("p").hide();
$("p").show();
```

(b) Toggle

```
$("p").toggle();
```

(c) Fade

```
$("p").fadeIn();
$("p").fadeOut();
```

(d) Slide

```
$("#panel").slideDown();
```

---

9. jQuery Animation

```
$("div").animate({
 left: '250px',
 opacity: '0.5'
});
```

---

10. jQuery DOM Manipulation

Methods:

- text() → get/set text
- html() → get/set HTML

- val() → get/set input value

👉 Example:

```
$("#btn").click(function(){
  $("#demo").text("Hello");
});
```

---

11. jQuery Advantages

- Simple and fast
- Less coding
- Powerful event handling
- Easy animations
- Works across browsers

---

12. Final Exam Answer (Short)

- jQuery is a **JavaScript library**
- Used for **DOM manipulation, events, effects, AJAX**
- Uses syntax: $(selector).action()
- Provides methods for **easy web development**